リレーシミュレーター

概要
画面構成
シナリオ
世界モデル
GUI（GUIエリア）
描画（描画エリア）
2つのプレイモード（将来的な発展形）
メモ：変数の相互関係・制約について
メモ：その他
メモ：参考

https://editor.p5js.org/74k4h170/full/dpAjr_Ndr

▼ 概要

リレーを実際にアニメーションできる。各種パラメータをGUIで設定してから、「再生」すると、画面上でリレーがシミュレーションされる。

- **experimentモード**：ユーザは各種パラメータを設定してから再生して眺める（適宜ノイズあり）。
- **gameモード**：passerの出走タイミングや、receiverの声かけタイミングなど、ユーザ自身がリアルタイム操作することもできる。

▼ 画面構成

- 描画エリア
  - 実際にリレーのシミュレーションを描画するエリア
  - 使用技術：p5.js
- GUIエリア
  - GUIで各種設定をするエリア
  - 使用技術：別のJSのUI系ライブラリ（p5.jsにあるGUIクラスなどは使わずに）

▼ シナリオ

1. ランナーの召喚（チームと区間を設定）

2. プレイモードを選択する（`experiment` or `game`）
3. 各ランナーのプロパティ設定（パラメータや初期値を設定）
4. 再生する（実際にシミュレーションする）

▼ 世界モデル

世界の時間

`update()`：世界を時間更新する

- `current_time += player.speed/60`
- 60fpsと再生速度を勘案して、シミュレーション世界内の時間を更新。
- 時間更新に応じて、runnerの各種プロパティを更新する。

runner（エージェント）

プロパティ

▼ `leg`: 走順（1 or 2 or 3 or 4）

▼ `team`: チーム（A or B or ... or H）

▼ `p`：位置[m]

▼ `pitch`：ピッチ[歩/秒]

=Hz周波数

▼ `stride`：歩幅[m]

※pitch * stride が速度になる。

▼ `pitch_dif`：ピッチの変化率[歩/秒^2]

▼ `stride_dif`：ストライドの変化率[m/秒]

▼ `phase`：腕振り位相ϕ[rad]（-π≤0≤π）

- 腕振りを単振動運動とみなす。

▼ `phase_v`：腕振り位相速度$\dot{\phi}$[rad/秒]

- + ⇒ 腕振り上げフェーズ
- - ⇒ 腕振り下ろしフェーズ

▼ `l`：腕長さ[m]※パラメータ

▼ `_is _running`：走り中かどうか（boolean）

▼ `_is_raised_arm`：バトンもらうための腕上げ中状態（boolean）

メソッド

▼ `phase_update()`：位相の更新

- 1・3走者（`leg == 1または3`）
- 「`_is_passing == true`かつ`phase≒0.5π`」なら、phase更新しない
- 「`_is_raised_arm == true`かつ`phase≒1.5π`」なら、`phase`更新しない
- そうでないとき`phase`更新
  - `phase += (player.speed*pitch/60)π`[rad]（60fpsでアニメーションの場合）
- 2・4走者なら（`leg == 2または4`）
  - 「`_is_passing == true`かつ`phase≒1.5π`」なら、`phase`更新しない
  - 「`_is_raised_arm == true`かつ`phas≒0.5π`」なら、`phase`更新しない
  - そうでないとき`phase`更新
    - `phase += (player.speed*pitch/60)π`[rad

（60fpsでアニメーションの場合）

▼ `p_update()`：位置の更新

- `p` += `pitch` * `stride` * `time_dif`

▼ `pitch_update()` pitchの更新（保留）

▼ `stride_update()` stride更新（保留）

▼ **バトンパス状態の更新関数（まだ仮。ひとまずテキトーでOK）**

- passerとreceiverを以下のように仮定義して、更新処理。
- たとえば現在走っているのがleg==1のひととleg==2のひとのとき、passerはleg==1のひと、receiverはleg==2のひとになる
- `receiver._is_raised_arm == true`、かつ、`receiver.p-passer.p ≤ passer.l*sin(passer.phase) + receiver.l`、かつ、passer.phase_v > 0（もしpasserが2走だったら、passer.phase_v ≤ 0）
- passer.phase == だったらというか位相速度と腕の長さlとが（パス可能距離）、
- などなど考え中

▼ `go()`：走り始める

- `._is _running = true`

▼ `call(_is_pass=true)`：声をかける

- `_is_pass == true` だったら、receiverの `raise_arm()` を発火（＝はい！）
- `_is_pass==false` だったら、receiverは急減速（＝待って！）

▼ `pass()`：バトン渡す

- どうにかこうにか

- 渡し終わると、`._is_passing`と、receiverの`_is_raised_arm`はfalse戻り、stop()。

▼ `raise_arm()`：バトンもらう構えをとる

- `_is_raised_arm = true`
- `stride *= 0.9` ストライドを下げる

▼ `lower_arm()`：バトンもらって腕をもどす

- `_is_raised_arm = false`
- `stride *= 1.0` ストライドを元にもどす

baton

▼ GUI（GUIエリア）

シナリオ1〜3について、各種ボタンやスライダ、チェックボックスなどで設定する。

ランナーの召喚

チームと区間を選ぶことで、その数だけランナーが初期位置に配置される。

- どのチームにする（A〜Hつまり1〜8レーン）？（複数選択可能チェックボックス）
  - A, B, C, D, E, F, G, H
- 区間はどうする？（チェックボックス）
  - 1-2間
  - 2-3間
  - 3-4間
  - 全区間

ゲームモード/実験モードの選択

チェックボックス（２択）

各ランナーごとのプロパティ設定

各ランナーの各種プロパティを設定。モードに応じて、GUIのパラメータ設定項目数が変わる。

初期値（=はデフォルト値）

- `p = 70`（スライダ）
- `pitch = 5.0`（スライダ）
- `stride = 2.0`（スライダ）
- `phase = 0`（スライダ）
- `_is_raised_arm = false`
- `_is_passing = false`

パラメータ

- `l = 0.8`

カメラ切り替えボタン群

- 1,2走のバトンゾーンにズームインする（ボタン）
- 2,3走バトンゾーンにズームインする（ボタン）
- 3,4走バトンゾーンにズームインする（ボタン）

再生設定

再生状態を保持（JSのvideoオブジェクトと同様な感じで、`player`オブジェクトにする？）

- `paused`：一時停止/再生中
- `current_time`：現在時間（）
- `speed`：再生速度（GUIのスライダorプルダウンで設定。x1.0で等倍速＝世界モデル内部の時間と現実の時間が一致する。x0.1〜x1.0まで0.1刻み？）
- `.play()`：再生する（スペースキー。トグル。）
- `.pause()`：一時停止する（スペースキー。トグル。）
- `.reset()`：一時停止して最初フレームへ
- （コマ戻し(左矢印キー)、コマ送り(右矢印キー)）

▼ 描画（描画エリア）

- 上から俯瞰した見た目を基本にする。
- ランナー（passer, receiver）とバトンの見た目
  - ランナー：直径0.6mの白塗り円
    - 1・3走はレーンの左側、2・4走はレーンの右側を走るようにする（現在はレーンの真ん中を走ってるけど）。
  - 腕振り：ランナー円の右端or左端にみよんみよん単振動のように伸び縮みする棒グラフ的な長方形。つまり、`runner.l * sin(runner.phase)` の長さ。
  - バトン：長さ0.3mの、黄緑色の棒
- パス可能状態では、passerとreceiverとバトンを目立つ色で描く
  - パス可能状態
- エリア左上に、シミュレーション世界内での現在時刻とバトン位置を表示する。

▼ 2つのプレイモード（将来的な発展形）

experimentモード

- いろんな問いを確かめるためにあるけど、中身はいったん保留でよし。
  - これこれこういう条件のときに、
    - 変数Xをどうすれば、タイムが良くなる？
    - バトンがゾーン内で渡るための変数Xの条件は？
    - など。
  - passerは声かけ「はい！」か「待って！」をどう使い分けているのか？
  - などなど
- 何千回のシミュレーション結果をグラフ描画
  - 縦軸・横軸でやる系はもちろん、
  - オリジナルなグラフ表現も楽しそう

gameモード

- 再生後に、ユーザが操作する
  1. receiverの出走タイミング（returnキー）
  2. passerの声かけタイミング（returnキー）
  3. receiverの腕上げタイミング（returnキー）
  4. passerのバトン差し出しタイミング（returnキー）

5. receiverのバトン引き戻しタイミング（returnキー）

- などなど

▼ メモ：変数の相互関係・制約について

- PとRの距離、pitch、stride、phase、Rの出走タイミング、Pの声かけタイミング、の総体的関係性。

💡

- これこれこういう条件のとき（初期値とパラメータ）、
- 変数Xがどうであれば（Xは複数変数の関係性でもOK）、
- パスは可能になるのか/最適になるのか？

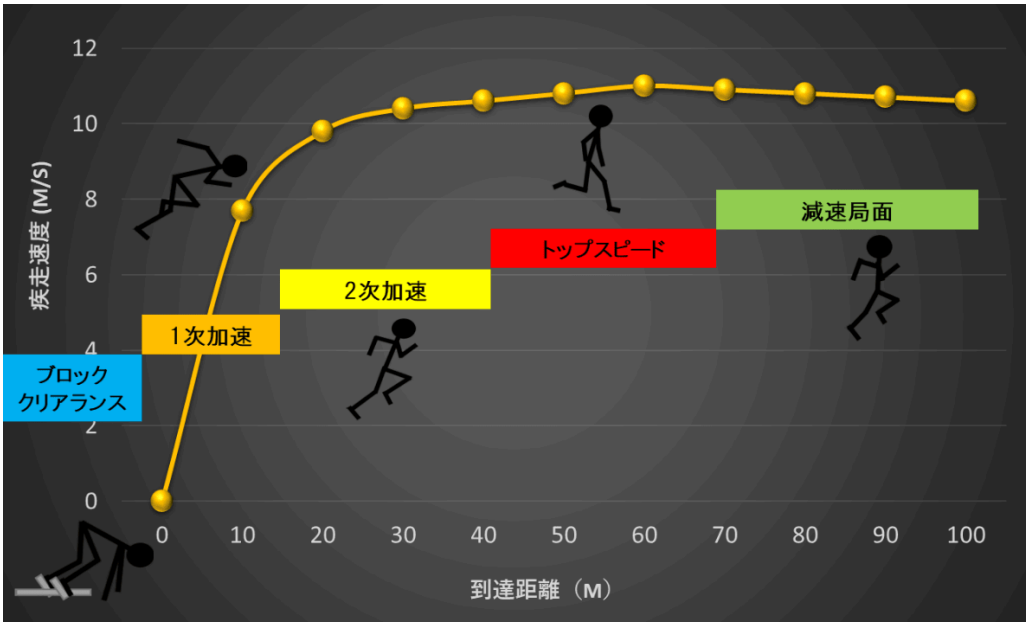
> 「まだみんな試してないだけで、実はこれこれこういう現実的な調整がありえて、それができればタイムは向上する」

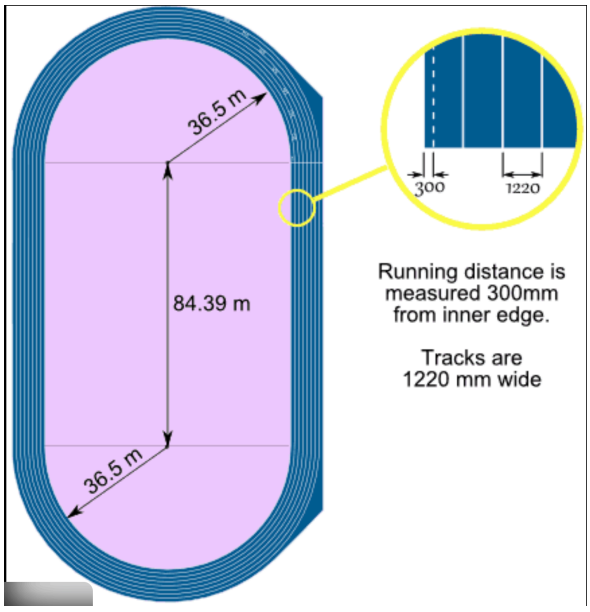
とかが導ければ、シミュレーションとしておもしろい。

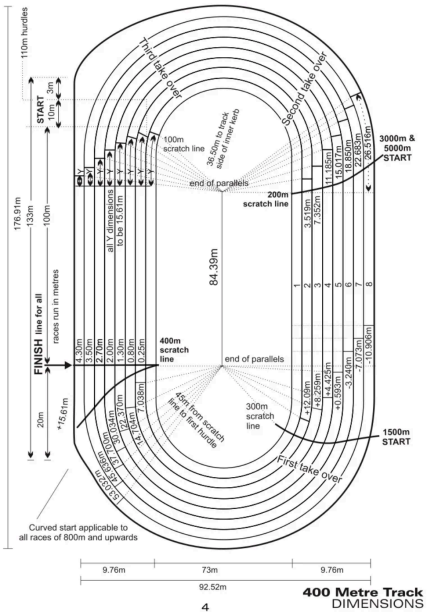
- 備考もろもろ
  - Rは「はい！」を聞いたら、「直近のphase=0.5πのタイミング」で腕を固定！
    - 自分の位相を無理やりずらしてすぐ`phase`=πに移行するわけではなさそう
  - もらい待ちの腕上げ状態ではstrideを90%とかにすべき？
  - ノイズいれたい（receiverの「早く出ちゃった！」的な）
  - `pitch`は4〜5Hz（minは疾走維持局面、maxは加速局面）
  - タウ関連
    - passerの声かけタイミングは、τで判断している、とかはできそう
      - τが大きくなったら「待って！」と言わざるを得ない
    - バトンがつまるときは、passerは渡す直前に「τドット一定調整」しているはず
    - 「τドット」と「二者の相対位相」の相互関係
      - passerは、「渡し始め」までのあいだで、receiverと`phase`をあわせようとしてたり？
      - receiverの背中をみて、τドット一定に保とうとするために、pitch*strideとphaseを調整している、といったことが？
  - 走りだしの`pitch`と`stride`の増減を近似せねば。
  - receiverが、バトンに触れたあとに、奪い去れるタイミングの条件は？
    - 踏みしめるタイミングじゃないとダメ、とか。

- 腕振りの位相とはなんなのか？（実映像だとなにで定義？末端？）
  - 「脚🦵」振りの位相は、古川流だったりするが・
  - 「腕💪」振りには「全振り」と「ちょい振り」がありえ、それをどう扱うか？
    - 「振幅のちがい」であって、位相変化様相については変わらずってことなのか？（例：passerが渡す直前は、体幹の横でちょい振りする）
    - ちょいぶりを「振幅の大小」とみなすならば、いっしょくたということになるが、「振幅が小さいほうが位相の急転換もやりやすい」みたいなこともある。
- 位相の範囲が絞られるということなのか？
- 4本手足の位相

▼ メモ：その他







- アンダーハンドパスとオーバーハンドパス
- 相手選手の存在
- 2ストライド未満にバトンパスを完了するのがよい (Zhao, 2025)サイレポ論文
- あるトップスピードで緩やかに減速しながら入ってくる。初期値のaは一定でOK。

1. 「はい！」
2. Rが、直近でphase=πになったとき、腕上げる（phase=πに固定）。
3. Pが、直近でphase=πになったとき、渡せそうな距離なら、腕を差し出す（phase=π）。
4. バトンが渡る（二人の腕の長さ）までこのままで、
5. バトンが渡れば、Pは用済み、Rはphase=π固定を終わらせる。

タウについて

共有ドキュメント

一人称モード、二人称モード、三人称モード

利得距離の議論（先行研究のシミュレーション研究にあり）

バトンパス時間を短さをYにする

Rの減速がないなら、バトンゾーンの後半で渡すほどよいはず？

▼ メモ：参考

https://cdn.shopify.com/s/files/1/1129/4180/files/400mAthleticsTrack_LineMarking.jpg?16182369258810024788=

https://www.mathsisfun.com/activity/images/athletics-track.gif